
task-app 30日間カリキュラム

Day 06

ユーザー登録画面を作ろう

Next.js 15 / TypeScript / Prisma / tRPC

目次

前回の振り返り	3
今日のゴール	3
始める前の前提	4
なぜこれを作るのか	5
実装ステップ一覧	8
Step 1: ページの土台を作る(読む目安: 5分)	9
Step 2: zodスキーマを定義する(読む目安: 7分)	13
Step 3: react-hook-formを設定する(読む目安: 5分)	18
Step 4: 名前・メール入力欄を作る(読む目安: 7分)	20
Step 5: パスワード入力欄を作る(読む目安: 5分)	25
Step 6: パスワード確認欄を作る(読む目安: 5分)	27
Step 7: tRPCで登録APIを呼ぶ(読む目安: 7分)	30
Step 8: アイコンとカード見出しを更新する(読む目安: 3分)	39
Step 9: エラー表示と送信ボタンを追加する(読む目安: 5分)	41
Step 10: ログインへのリンクを追加して完成(読む目安: 3分)	44
完成コード全体	57
Day 05(ログイン)と Day 06(登録)の比較	71
今日のまとめ	71
つまずきポイント	72
今日学んだ用語	74
応用課題: 確認欄のエラーを出す条件を変える	74
理解チェック	75
次回予告	76
次に読むもの	76
奥付	76

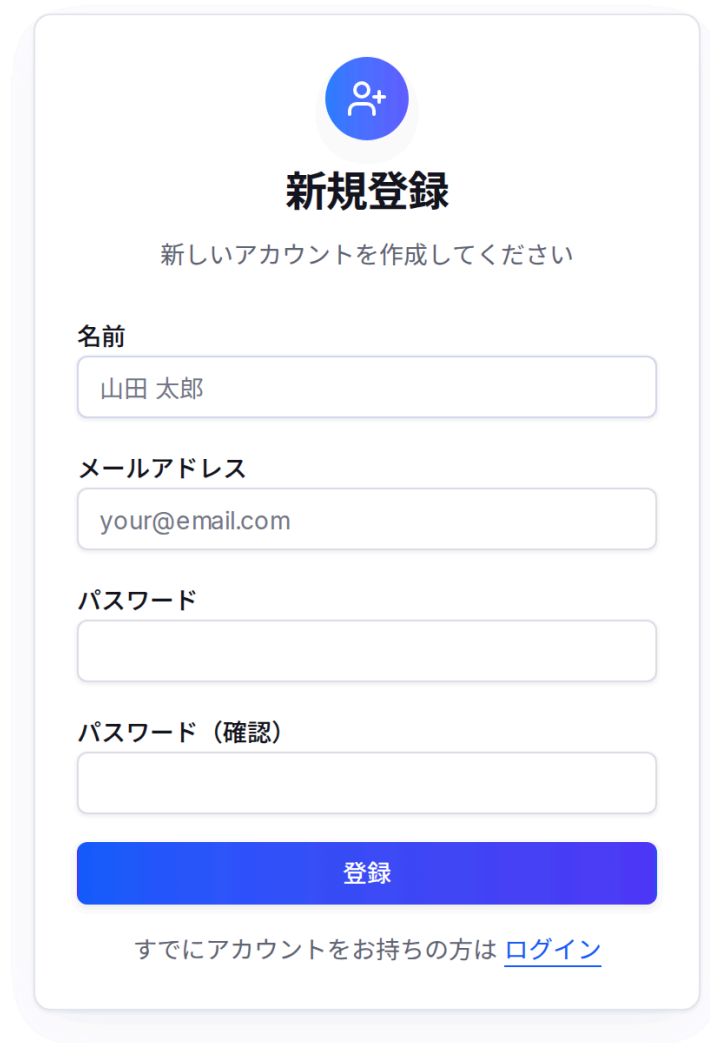
前回の振り返り

Day 05 では react-hook-form と zod を使ってバリデーション付きのログイン画面を作りました。フォーム管理とバリデーションの基本パターンを習得したので今日はその応用に進みます。

今日のゴール

Day 05 で学んだ react-hook-form + zod パターンを応用してユーザー登録画面を作ります。パスワード確認チェックをはじめ、より高度なバリデーションに挑戦します。

スクリーンショット: 完成したユーザー登録画面(名前・メール・パスワード入力欄がある状態)



The image shows a 'New Registration' (新規登録) form. At the top is a blue circular icon with a white person silhouette and a plus sign. Below the icon is the title '新規登録' in bold black text, followed by the instruction '新しいアカウントを作成してください' in a smaller font. The form contains four input fields: '名前' (Name) with the value '山田 太郎', 'メールアドレス' (Email Address) with the value 'your@email.com', 'パスワード' (Password), and 'パスワード (確認)' (Password (Confirmation)). Below these fields is a prominent blue button labeled '登録' (Register). At the bottom, there is a link that says 'すでにアカウントをお持ちの方は [ログイン](#)' (If you already have an account, please [login](#)).

図1: 新規登録画面。名前・メールアドレス・パスワード・パスワード(確認)の4つの入力欄と登録ボタン、ログイン画面へのリンクが並んでいる

始める前の前提

- Day 05 のログイン画面を作り終えています。
- `src/app/register/page.tsx` を新規作成できます。
- `react-hook-form` と `zod` の基本形を見たことがあります。
- 登録 API のサーバー内部は Day 07 で確認します。今日はフォーム作成と送信体験に集中します。

なぜこれを作るのか

ログインするにはまずアカウントが必要です。登録画面ではパスワードの確認入力や複数フィールドをまたぐバリデーションといった、実務でよく使うテクニックを学びます。

例え話: ユーザー登録は「会員証の申込書」を書く手続きです。名前と連絡先を記入して暗証番号(パスワード)を決めます。「もう一度暗証番号を書いてください」と確認するのは銀行の暗証番号設定と同じ仕組みです。

登録処理のフロー

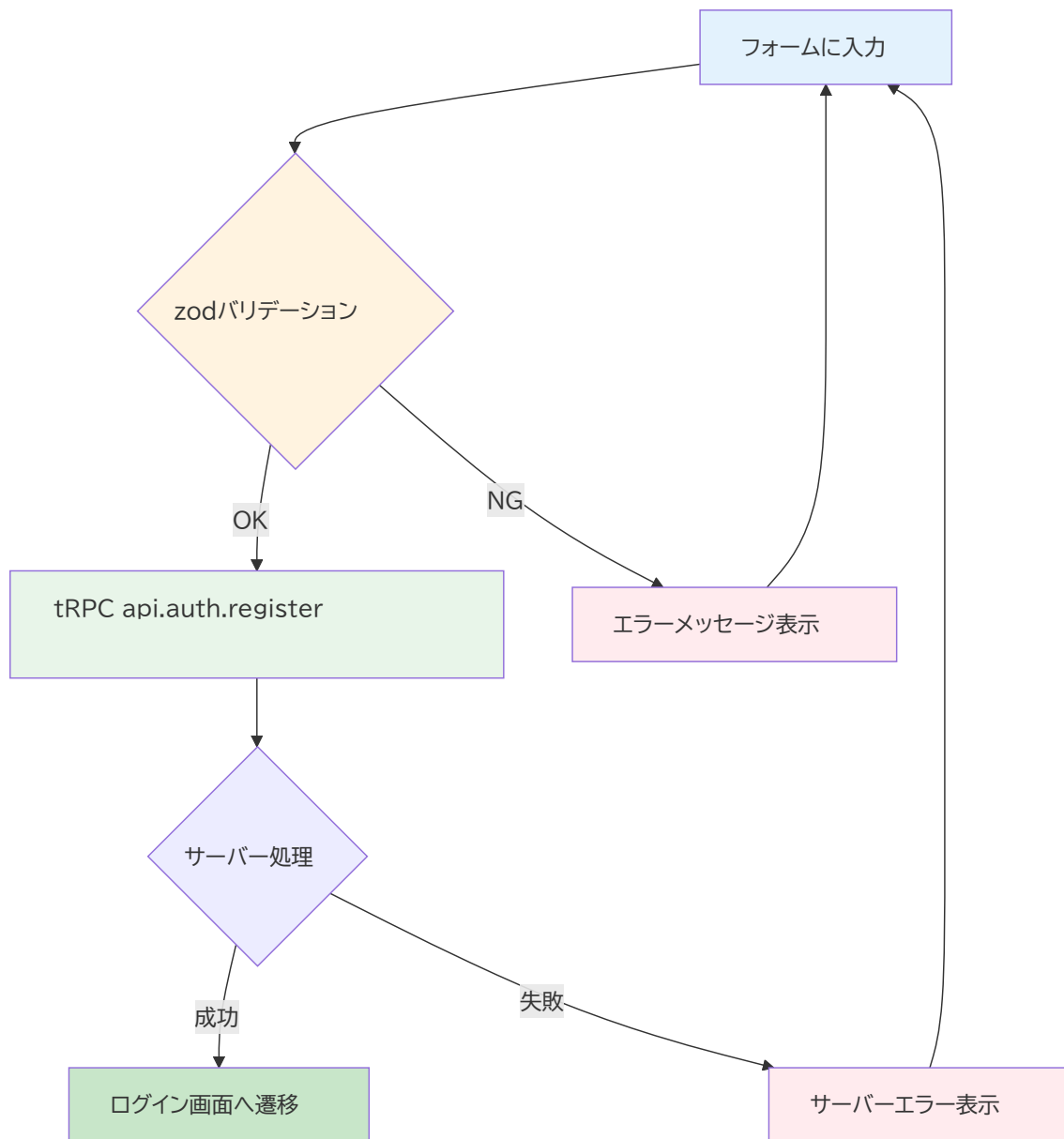


図2: 登録処理のフロー

この図で目を留めてほしいのは「zodバリデーション」の菱形から矢印が2本出ているところです。zod の検査で入力に誤りが見つかったら「エラーメッセージ表示」へ進み、`api.auth.register` を呼び出しません。画面側のバリデーションは送信ボタンとサーバーの間に立って通信そのものを止めています。

ただしこの関門はブラウザの中にしかありません。通信を自分で組み立てて送れば、画面側の検査を通さずにサーバーへ登録を要求できるので Day 07 で読むサーバー側でも同じ検査をやり直します。画面側は入力ミスをその場で知らせるため、サーバー側は不正な登録を通さないためにあります。役割が違うので片方があればもう片方はいらない、とはなりません。

やること / やらないこと

やること	やらないこと
react-hook-form + zod で登録フォーム	useState で個別管理
<code>.refine()</code> でパスワード一致チェック	手動で if 文比較
tRPC で登録API呼び出し	サーバー側の処理(Day 07)
shadcn/ui でカードデザイン	CSS のゼロからの設計

新しく学ぶ概念

概念	読み方	役割	例え
<code>.refine()</code>	リファイン	複数フィールドを またぐカスタム チェック	会員証の申込書で「暗証番号」と「確認欄」が一致しているか、受付係が見比べる チェック

実装ステップ一覧

ステップ	作業内容	読む時間の目安
Step 1	ページの土台を作る (import 含む)	5 分
Step 2	zod スキーマを定義する	7 分
Step 3	react-hook-form を設定する	5 分
Step 4	名前・メール入力欄を作る	7 分
Step 5	パスワード入力欄を作る	5 分
Step 6	パスワード確認欄を作る	5 分
Step 7	tRPC で登録 API を呼ぶ	7 分
Step 8	アイコンとカード見出しを更新する	3 分
Step 9	エラー表示と送信ボタンを追加する	5 分
Step 10	ログインへのリンクを追加して完成	3 分

読む時間の合計(仮): 約 52 分です。

表と各 Step に記した時間は、説明とコードを読む時間の仮の目安です。実測した値ではありません。コードの入力、動作確認、ダウンロードや起動の待ち時間、調べものには別に時間を取ってください。

Step 1: ページの土台を作る(読む目安: 5分)

ゴール: 登録ページの基本ファイルを作成し、必要な import をすべて記述します。

今回は最初に必要な import をまとめて書きます。Step ごとに何度もファイル先頭に戻る手間を省くためです。

実装:

`src/app/register/page.tsx` を新規作成し、以下のコードを書いてください。

```
// filepath: src/app/register/page.tsx
// クライアントコンポーネント宣言と全import
'use client';

import { zodResolver } from '@hookform/resolvers/zod';
import { AlertCircle, UserPlus } from 'lucide-react';
import Link from 'next/link';
import { useRouter } from 'next/navigation';
import { useState } from 'react';
import { useForm } from 'react-hook-form';
import { z } from 'zod';
```

先頭の `'use client'` はこのファイルをブラウザ側で動かす宣言です。登録フォームは1文字打つたびに入力値を持ち回るのでサーバーだけで完結する部品にはできません。続く取り込みはこれから使う道具を先に並べておく宣言です。

`zodResolver` は zod のルールを react-hook-form につなぐ接続部品、

Day 06: ユーザー登録画面を作ろう

`useForm` はフォーム本体、`z` は入力ルールを書くための道具です。`useRouter` は登録が終わった後の画面移動、`useState` はサーバーから返ったエラー文を覚えておく場所に使います。

確認ポイント:

- `src/app/register/page.tsx` を保存しました。
- この時点で `/register` を開くと赤いエラー画面になります(ページ本体をまだ書いていないためで、写し間違いではありません)。
- ブラウザで開くのはこのあと3つ目のブロックまで書き終えてからにしてください。

続いてimportの下に shadcn/ui コンポーネントと tRPC の import を追加します。

```
// filepath: src/app/register/page.tsx
// 上の import の続きに追加
import {
  Alert, AlertDescription, AlertTitle,
} from '@component/ui/alert';
import { Button } from '@component/ui/button';
import {
  Card, CardContent,
  CardDescription, CardHeader, CardTitle,
} from '@component/ui/card';
import { Input } from '@component/ui/input';
import { Label } from '@component/ui/label';
import { api } from '@trpc/react';
```

2枚目は画面に置く見た目の部品です。`Card` が枠、`Label` と `Input` が見出しと入力欄、`Button` が送信ボタン、`Alert` はサーバーが登録を断ったときに理由を赤い帯で見せる部品です。最後の `api` は tRPC の窓口で、Step 7 でサーバーの登録処理を呼ぶときに使います。いまはどれも使っていないのでエディタが「未使用」の警告を出しますが Step 10 までに全部出番が来ます。

確認ポイント:

- ファイルを保存しました。
- `npm run dev` を動かしているターミナルに赤いエラーが出ていません(ブラウザはまだ開かなくてよいです)。

import パスについて: このプロジェクトでは `@/component/ui/...` を使います。他の教材やドキュメントでは `@/components/ui/...` と複数形を使う場合もありますが実際のプロジェクト構成に合わせてください。

次にページ本体のコンポーネントを定義します。import の下に追加してください。

Day 06: ユーザー登録画面を作ろう

```
// filepath: src/app/register/page.tsx
// import群の下に追加(ページ本体)
export default function RegisterPage() {
  return (
    <div className="flex min-h-screen
      items-center justify-center px-4">
      <Card className="w-full max-w-sm">
        <CardHeader>
          <CardTitle>新規登録</CardTitle>
        </CardHeader>
      </Card>
    </div>
  );
}
```

まずは中身のないカードだけを置きます。外側の `<div>` に付けた `min-h-screen` が画面の高さいっぱいを使う指定で、`items-center justify-center` がその中でカードを縦横の中央へ寄せます。`max-w-sm` を付けているので大きなモニターでもカードは横に間延びしません。フォームの中身はこの `<Card>` の内側へ Step 4 から1欄ずつ足していきます。

確認ポイント:

- `src/app/register/page.tsx` を保存しました。
- `npm run dev` でエラーが出ていません。
- ブラウザで `/register` にアクセスして「新規登録」カードが表示されます。

Step 2: zodスキーマを定義する(読む目安: 7分)

ゴール: パスワード確認チェック付きのバリデーションスキーマを作ります。

例えば: `.refine()` は会員証の申込書で各欄を個別にチェックした後にやる「最終確認」です。各項目が正しくても「暗証番号」と「暗証番号(確認)」が一致していなければ受理されません。受付係が最後に2つの欄を見比べる作業が `.refine()` です。

実装:

`export default function RegisterPage()` の上(import群と関数定義の間)に以下を追加します。

Day 06: ユーザー登録画面を作ろう

```
// filepath: src/app/register/page.tsx
// import群の下、RegisterPage関数の上に追加
const registerSchema = z.object({
  name: z.string()
    .min(1, '名前を入力してください'),
  email: z.string()
    .email('有効なメールアドレスを入力してください'),
  password: z.string()
    .min(8, 'パスワードは8文字以上で入力してください')
    .regex(/[A-Z]/,
      'パスワードに大文字を1文字以上含めてください')
    .regex(/[a-z]/,
      'パスワードに小文字を1文字以上含めてください')
    .regex(/[0-9]/,
      'パスワードに数字を1文字以上含めてください')
    .regex(/[^\A-Za-z0-9]/,
      'パスワードに記号を1文字以上含めてください'),
  confirmPassword: z.string()
    .min(1, 'パスワード(確認)を入力してください'),
}).refine(
  (data) => data.password === data.confirmPassword,
  {
    message: 'パスワードが一致しません',
    path: ['confirmPassword'],
  }
);
```

このブロックは括弧を開いたまま終わっているので保存すると赤い波線が出ます。それで正常です。閉じるのは次のブロックなので先に中身を読んでおきましょう。

1行ずつが「これはブラウザで止める」という宣言です。`name` の `.min(1)` を外すと名前が空のままでも送信ボタンが通り、断られると分かっているリクエストがサーバーへ飛びます。`email` の `.email()` を外すと `abc` のような文字列でも送信され、間違いを知るのは通信の返事が返ってきたあとになります。`password` に並ぶ4本の `.regex()` は大文字・小文字・数字・記号をそれぞれ1文字以上求めます。この4つの文字種の条件は Day 07 で読むサーバー側の `auth.ts` もチェックします。サーバー側には UTF-8(文字をバイト列に変換する形式)で72バイト以内という追加の条件もあります。`bcrypt`(パスワードを元に戻せない形へ変換する道具)が扱える長さの上限です。今日の画面はこの長さをまだ検査しないので、超えた場合は Step 9 の赤い帯にサーバーからのエラーが出ます。登録を最終的に許すかどうかを決めるのはサーバー側です。ブラウザ側のこのスキーマが受け持つのは打ち終わったその場で間違いを知らせることと、通らない通信を減らすことの2つです。最後の `.refine()` は項目ごとのチェックが済んだあとに、2つの欄をまたいで見る確認です。項目側でエラーが出ていても続けられる種類のエラーなら `.refine()` も走ります。短すぎるパスワードで確認欄も違っていれば両方のエラーが同時に出来ます。

```
// filepath: src/app/register/page.tsx(同じファイルの続き)
  },
);
```

閉じ括弧の2行で `registerSchema` が完成します。`.refine()` は `z.object({...})` の後ろに付ける後付けの検査なので閉じ括弧も `.refine()` の引数側と、その外側の2段になります。片方を書き忘れると `registerSchema` が式の途中と見なされ、次に書く型定義まで巻き込んでエラーが出ます。

Day 06: ユーザー登録画面を作ろう

確認ポイント:

- ファイルを保存しました。
- `npm run dev` でエラーが出ていません。
- この時点では画面に変化はありません。Step 4 でフォームを作るところで定義したルールが動き出します。

続いてスキーマの直下に型定義を追加します。

```
// filepath: src/app/register/page.tsx
// registerSchemaの直下に追加
type RegisterFormData =
  z.infer<typeof registerSchema>;
```

`z.infer` はいま書いたルールから TypeScript の型を作り出す道具です。`name` は文字列、`email` も文字列、`confirmPassword` も文字列、という形が自動で決まります。同じ内容を型として手で書き直してもかまいませんがそうするとルールを1本足したときに型だけ古いまま取り残されます。`z.infer` にしておけばスキーマが唯一の正解になり、両者が食い違いません。

確認ポイント:

- `registerSchema` に `.refine()` が含まれています。
- `RegisterFormData` 型が定義されています。
- `npm run dev` でエラーが出ていません。

バリデーションルール一覧

フィールド	ルール	エラーメッセージ
名前	必須(1文字以上)	名前を入力してください
メール	必須 + メール形式	有効なメールアドレスを入力してください
パスワード	必須 + 8文字以上 + 大文字・小文字・数字・特殊文字を含む	各要件ごとのエラーメッセージ
パスワード確認	必須 + パスワードと一致	パスワードが一致しません

パスワード強度要件: 今日の画面では最低8文字と4つの文字種を検査します。この5つはサーバー側も検査します。サーバー側だけが検査する72バイトの上限を超えた入力、送信後に Step 9 の赤い帯で知らせます。

サーバー側パスワードバリデーション一覧

要件	内容	クライアント側と一致するか
最低文字数	8文字以上	☒ 一致
大文字	1文字以上含む (/[A-Z]/)	☒ 一致
小文字	1文字以上含む (/[a-z]/)	☒ 一致

要件	内容	クライアント側と一致するか
数字	1文字以上含む (<code>/[0-9]/</code>)	☒ 一致
特殊文字	1文字以上含む (<code>/[^\A-Za-z0-9]/</code>)	☒ 一致
上限バイト数	UTF-8で72バイト以内	今日の画面では未検査。 超えると Step 9 の赤い帯にエラーを表示

`.refine()` のコード解説

コード	意味	例え
<code>.refine(fn, opts)</code>	カスタムバリデーションを追加	申込書の最終チェック項目を追加する
<code>data.password === data.confirmPassword</code>	2つのフィールドを比較	暗証番号の2つの欄を見比べる
<code>path: ['confirmPassword']</code>	エラーの表示先を指定	「確認欄」に赤線を引く

Step 3: react-hook-formを設定する(読む目安: 5分)

ゴール: `useForm` と `zodResolver` を組み合わせて Zod スキーマによるバリデーションをフォームに接続します。

実装:

`export default function RegisterPage()` の `{` の直後(`return` の前)に以下を追加します。

```
// filepath: src/app/register/page.tsx
// RegisterPage() { の直後、return の前に追加
const {
  register,
  handleSubmit,
  formState: { errors },
} = useForm<RegisterFormData>({
  resolver: zodResolver(registerSchema),
});

// 仮の送信処理(Step 7で書き換えます)
// 動作確認用の一時コードです。Step 7で必ず削除してください。
const onSubmit = async (
  data: RegisterFormData
) => {
  console.log('登録データ:', data);
};
```

`useForm` から受け取る3つにはそれぞれ役目があります。`register` は入力欄をフォームにつなぐ関数、`handleSubmit` は送信時にまずバリデーションを走らせる関数、`errors` はどの欄がどのルールに引っかかったかを持つ入れ物です。`resolver` に `zodResolver(registerSchema)` を渡した時点で、Step 2 で書いたルールがこのフォームの検査役になります。

Day 06: ユーザー登録画面を作ろう

下の `onSubmit` は Step 7 まで使う仮置きで、受け取った値をコンソールに出すだけです。本物の送信処理を先に書くと入力欄がまだ1つも無い状態でサーバーを呼び出すことになります。フォームが組み上がるまでは値がどんな形で届くかを目で見えて確かめる段階にしておきます。

確認ポイント:

- ファイルを保存しました。
- `npm run dev` でエラーが出ていません。
- ブラウザで `/register` にアクセスしてエラーが出ていません。
- この時点ではまだフォーム欄は表示されません。Step 4 で入力欄を作ります。
- `console.log` は動作確認後に残さず、Step 7 で必ず削除します。

`useForm<RegisterFormData>` の型引数に Zod スキーマから推論した型を渡すことで、入力値の型が保証されます。

`zodResolver(registerSchema)` が送信前に自動でバリデーションを実行し、エラーは `formState.errors` に格納されます。フォームの項目が増えても `useForm` の呼び出し自体は変わりません。

Step 4: 名前・メール入力欄を作る(読む目安: 7分)

ゴール: 名前とメールアドレスの入力欄をフォームに追加します。

実装:

`return` 文の中にある `<CardHeader>...</CardHeader>` の直後に `<CardContent>` を追加します。

```

{/* filepath: src/app/register/page.tsx */}
{/* </CardHeader> の直後に追加 */}
<CardContent>
  <form onSubmit={handleSubmit(onSubmit)}
    className="space-y-4">
    <div className="space-y-2">
      <Label htmlFor="name">名前</Label>
      <Input
        id="name"
        type="text"
        placeholder="山田 太郎"
        autoComplete="name"
        autoFocus
        {...register('name')}
      />
      {errors.name && (
        <p className="text-sm text-destructive">
          {errors.name.message}
        </p>
      )}
    </div>
  </form>
</CardContent>

```

`{...register('name')}` がこの `<Input>` を `name` という名前でフォームにつなぐ書き方です。ここを書き忘れると見た目は入力欄でも打った文字がフォームに届きません。何を入力しても送信データに `name` が入らず、`errors.name` に必須エラーが出たまま止まります。

Day 06: ユーザー登録画面を作ろう

`{errors.name && (...)}` は `errors.name` に中身があるときだけ赤い文字を出す書き方です。エラーが無いときの `errors.name` は `undefined` なので `<p>` そのものが描かれません。だから正常時に空の行が残ってボタンの位置がずれることもありません。

確認ポイント:

- ファイルを保存しました。
- ブラウザに名前入力欄が表示されています。

次に `<form>` の中で名前入力欄(`</div>` の後)にメール入力欄を追加します。

```

{/* filepath: src/app/register/page.tsx */}
{/* <form>内、名前入力欄の </div> の後に追加 */}
<div className="space-y-2">
  <Label htmlFor="email">
    メールアドレス
  </Label>
  <Input
    id="email"
    type="email"
    placeholder="your@email.com"
    autoComplete="email"
    {...register('email')}
  />
  {errors.email && (
    <p className="text-sm text-destructive">
      {errors.email.message}
    </p>
  )}
</div>

```

`type="email"` はスマートフォンでメール入力用のキーボードを使うための指定です。ブラウザによる形式検査も働きます。たとえば `abc` と入力して送信すると先にブラウザの案内が出るため Zod の検証へは進みません。

ブラウザの検査を通った入力を、Step 2 の `.email()` がさらに確かめます。こちらで失敗した場合は指定した日本語のメッセージが出ます。ブラウザ側の案内は端末や言語設定によって変わります。

Day 06: ユーザー登録画面を作ろう

`register('email')` に渡す文字列は `registerSchema` のキー名と必ずそろえます。ここを `mail` と書き間違えると `RegisterFormData` にそんな項目は無いと TypeScript がその場で赤線を引いてくれます。Step 2 で型をスキーマから作っておいた効き目がここで出ます。

確認ポイント:

- 名前欄とメール欄が表示されています。
- ここまでのコードの綴りと括弧の対応を確認しました。

送信を試すのは Step 9 からです。フォームを送るボタンはまだ書いていません。

スクリーンショット: 下の画像は Step 9 まで書き終えた完成後の画面です。赤枠の中がこの Step で足した名前とメールの 2 欄です。いまの自分の画面にはその下のパスワードとパスワード(確認)の欄がまだ出ていません。パスワード欄は Step 5、パスワード(確認)欄は Step 6 で足します。

新規登録

新しいアカウントを作成してください

① 名前 *

山田 太郎

② メールアドレス *

your@email.com

パスワード *

8文字以上で、大文字・小文字・数字・特殊文字を含め、UTF-8で72バイト以内にしてください

パスワード (確認) *

登録

すでにアカウントをお持ちの方は [ログイン](#)

図3: 完成後の新規登録画面。赤枠①が名前の欄、赤枠②がメールアドレスの欄

Step 5: パスワード入力欄を作る(読む目安: 5分)

ゴール: パスワード入力欄を追加します。

実装:

`<form>` の中で、メール入力欄の `</div>` の後に追加します。

```
{/* filepath: src/app/register/page.tsx */}
{/* <form>内、メール入力欄の </div> の後に追加 */}
<div className="space-y-2">
  <Label htmlFor="password">
    パスワード
  </Label>
  <Input
    id="password"
    type="password"
    autoComplete="new-password"
    {...register('password')}
  />
  {errors.password && (
    <p className="text-sm text-destructive">
      {errors.password.message}
    </p>
  )}
</div>
```

`type="password"` にすると打った文字が黒丸に置き換わります。ここを `type="text"` のままにすると入力中のパスワードがそのまま画面に出ます。カフェで肩越しに見られる場面だけでなく、画面共有や操作動画にも残ります。表示を隠すのは見た目の好みではなく、目に触れる経路をふさぐためです。

エラー表示の形は名前欄と変わりませんが読むのは `errors.password` です。Step 2 で `.regex()` を4本つないだので条件を満たさない間は先に引っかけたルールメッセージが1つだけ出ます。全部まとめて出ないぶん、読者は次に直す点を1つずつ追えます。

確認ポイント:

- パスワード欄が表示され、打った文字が黒丸になります。
- ここまでのコードの綴りと括弧の対応を確認しました。

8文字未満で弾かれることは送信ボタンがそろそろ Step 9 で確かめます。

`autocomplete="new-password"` を指定するとブラウザが「新しいパスワードの入力欄」と認識します。既存パスワードの自動入力を防げます。

Step 6: パスワード確認欄を作る(読む目安: 5分)

ゴール: パスワード確認入力欄を追加し、一致チェックを動作させます。

会員証の申込書で「暗証番号を2回書いてください」と求められるのと同じです。Step 2 で `.refine()` を設定済みなのでコードを追加するだけで自動的にチェックされます。

実装:

`<form>` の中で、パスワード入力欄の `</div>` の後に追加します。

```
{/* filepath: src/app/register/page.tsx */}
{/* <form>内、パスワード入力欄の </div> の後に追加 */}
<div className="space-y-2">
  <Label htmlFor="confirmPassword">
    パスワード(確認)
  </Label>
  <Input
    id="confirmPassword"
    type="password"
    autoComplete="new-password"
    {...register('confirmPassword')}
  />
  {errors.confirmPassword && (
    <p className="text-sm text-destructive">
      {errors.confirmPassword.message}
    </p>
  )}
</div>
```

この欄は空欄チェックに加えて、パスワード欄との比較でも赤くなります。

`.refine()` に書いた `path: ['confirmPassword']` が「パスワードが一致しません」の行き先をこの欄に決めているからです。

`path` を書かなかった場合エラーはフォーム全体に付きます。

`errors.confirmPassword` は空のままなのでどの入力欄の下にも赤い文字が出ません。送信ボタンがそろった Step 9 以降で見えるのは「登録ボタンを押しても画面が何も変わらない」状態だけになり、どこを直せばよいのか手掛かりがありません。エラーは出すだけでなく、出す場所まで指定して初めて役に立ちます。

確認ポイント:

- パスワード確認欄が表示されています。
- ここまでのコードの綴りと括弧の対応を確認しました。

2つのパスワードが食い違うときの動きは送信ボタンがそろそろ Step 9 で確かめます。

スクリーンショット: 下の画像は Step 9 まで書き終えた完成後の画面です。赤枠①は Step 5 で足したパスワード欄、赤枠②がこの Step で足したパスワード(確認)欄です。これで入力欄は4つそろいます。

新規登録

新しいアカウントを作成してください

名前 *

山田 太郎

メールアドレス *

your@email.com

① パスワード *

8文字以上で、大文字・小文字・数字・特殊文字を含め、UTF-8で72バイト以内にしてください

② パスワード (確認) *

登録

すでにアカウントをお持ちの方は [ログイン](#)

図4: 完成後の新規登録画面。赤枠①がパスワードの欄、赤枠②がパスワード(確認)の欄

Step 7: tRPCで登録APIを呼ぶ(読む目安: 7分)

ゴール: 登録ボタンを押したらサーバーにユーザー情報を送信します。

コードの順序: `router` と `error` の state は `useForm` より前に定義します。これは実際の `src/app/register/page.tsx` の構造と一致しています。最終的な定義順は `router` → `error state` → `useForm` → `registerMutation` になります。

実装:

`RegisterPage()` の中で、Step 3 で書いた `useForm` の設定の前(関数の先頭)に以下を追加します。

```
// filepath: src/app/register/page.tsx
// RegisterPage内、useFormの設定の前に追加
const router = useRouter();
const [error, setError] =
  useState<string | null>(null);
```

続いてStep 3 で書いた `useForm` の設定の閉じ括弧 `});` の後(仮の `onSubmit` の前)に、以下を追加します。

```
// filepath: src/app/register/page.tsx
// useFormの設定の後、仮のonSubmitの前に追加
// trRPCの登録API呼び出し
const registerMutation =
  api.auth.register.useMutation({
    onSuccess: () => {
      router.push('/login?registered=1');
    },
    onError: (error) => {
      setError(
        error.message ??
        'ユーザー登録中にエラーが発生しました'
      );
    },
  });
```

`registerMutation` はサーバーの登録処理を呼ぶための取っ手です。Day 09以降で使う `useQuery` は置いただけで自動的に走りますが `useMutation` は走りません。データを書き換える処理を画面を開いた勢いで実行されては困るので `mutate(...)` を呼ぶまで待ちます。

`onSuccess` は登録が通ったときに走ります。登録のあとにログインも試せるように、移動先はログイン画面にしています。ログインの状態を決めるサーバー側の処理はDay 07 で確認します。`registered=1` は、この画面移動で完了案内を出すための固定マーカー(決め打ちの目印)です。URLは利用者が書き換えられるので、登録済みであることの証明には使えません。認証・権限・データの判定はサーバー側で行います。URLに案内文そのものを入れないので、書き換えられた文字を画面へ表示することはありません。

ログイン画面にも、この固定フラグを読む処理を追加します。`LoginForm` にある `callbackUrl` の定義の直後へ次の2行を追加してください。

```
// filepath: src/app/login/page.tsx(callbackUrl の直後に追加)
const registrationCompleted =
  searchParams?.get('registered') === '1';
```

比較する値を `'1'` に限定しているのがポイントで

す。`?registered=登録しました` のような値を受け取っても、その中身を画面へ出しません。

`<form>` の開始タグの直後、既存の `{error && (` より前へ案内を追加します。

```
{/* filepath: src/app/login/page.tsx(form の直後に追加)
*/}
{registrationCompleted && (
  <Alert role="status">
    <AlertTitle>登録が完了しました</AlertTitle>
    <AlertDescription>
      ログインしてください。
    </AlertDescription>
  </Alert>
)}
```

`role="status"` は、操作結果の案内を読み上げソフトへ伝える目印です。赤いエラー帯とは役割が違うため、`variant="destructive"` は付けません。

Day 06: ユーザー登録画面を作ろう

`onError` はサーバーが登録を断ったときに走り、その理由を `error` state へ入れます。表示するのは Step 9 なので、いまは受け取る場所までです。

確認ポイント:

- 登録成功時の移動先が `/login?registered=1` になっています。
- ログイン画面が `registered` の値を文章として表示していません。
- `npm run dev` でエラーが出ていません。

続いて Step 3 で書いた仮の `onSubmit` を削除します。2 行のコメントから `const onSubmit = async (` に対応する `};` までを、次の形のままとめて削除してください。

```
// 仮の送信処理(Step 7で書き換えます)
// 動作確認用の一時コードです。Step 7で必ず削除してください。
const onSubmit = async (
  data: RegisterFormData
) => {
  console.log('登録データ:', data);
};
```

この仮実装を残したまま公開すると登録ボタンが押されるたびに、入力されたパスワードが開発者ツールのコンソールへそのまま並びます。読者本人の画面にはしか出ないとはいえ、操作動画やサポート用のスクリーンショットには写ります。仮実装は消すところまでが 1 組だと考えて手元の練習でも必ず削除します。

この `console.log` は Step 3 の仮実装です。確認が終わったら残さず削除し、下の `registerMutation` を使う本実装に置き換えてください。

削除したら以下の新しい `onSubmit` を追加します。

```
// filepath: src/app/register/page.tsx
// Step 3の仮のonSubmitを書き換え
const onSubmit = async (
  data: RegisterFormData
) => {
  setError(null);
  // confirmPasswordはサーバーに送らない
  registerMutation.mutate({
    name: data.name,
    email: data.email,
    password: data.password,
  });
};
```

先頭の `setError(null)` は前回の失敗で出した赤い帯を消してから送るための1行です。これが無いと2回目の送信が成功しても古いエラー表示が画面に残り、成功したのに失敗したように見えます。

`mutate` に渡しているのは `name`・`email`・`password` の3つで、`confirmPassword` は入っていません。確認欄は入力ミスをその場でを見つけるための欄なのでサーバーまで運ぶ意味がないからです。送る項目は少ないほど途中で漏れる範囲も狭くなります。

確認ポイント:

- `registerMutation` が定義されています。
- `onSubmit` で `confirmPassword` を除外して送信しています。
- `npm run dev` でエラーが出ていません。

`confirmPassword` はフロントエンド専用のフィールドです。会員証の申込書で暗証番号を2回書くのは記入ミスをその場で見つけるためです。受付窓口(サーバー)に渡すのは `name`, `email`, `password` の3つだけです。

送ったパスワードはサーバーでどうなるか

`mutate` で送ったパスワードはサーバーに届いた時点では読める文字列のままです。いま送り先にしている `http://localhost:3000` は同じパソコンの中で完結する宛先なので、通常はパソコンの外のネットワークへ出ません。ただし同じパソコン内でも、OS 上の権限を持つプログラムやブラウザの開発者ツールから読まれる可能性はあります。Day 04 で公開した `https://` の本番環境なら、パソコンからサーバーまでの区間はすでに暗号化されています。Day 30 で仕上げて公開するときも、この区間は暗号化された通信です。どちらの場合も受け取ったサーバーの手元では読める文字列に戻ります。これをそのままデータベースの列に保存すると中身が一度でも外へ出たとき全員のパスワードが読める形で並びます。同じパスワードを別のサービスでも使っている人はそちらまで芋づるで入られます。

だから Day 07 で読むサーバー側は、受け取った文字列を `bcrypt`(元に戻せない形へ変換する道具)に通します。Day 07 の登録処理では、`bcrypt.hash` がソルト(変換時に加えるランダムな値)を毎回新しく作ります。そのため、同じパスワードを登録しても、変換後の文字列は通常異なります。データベースには、ソルトと計算回数の設定を含む変換後の文字列だけを保存します。

ログインでは、`bcrypt.compare` に入力されたパスワードと保存済みの文字列を渡します。`bcrypt.compare` は、保存済みの文字列に含まれるソルトと計算回数の設定を使って一致するか確かめます。新しいソルトで別のハッシュを作り、文字列同士を比べる処理ではありません。保存済みの文字列から元のパスワードを復元する処理でもありません。ただし弱いパスワードは、候補を順番に試す総当たりや辞書攻撃で推測される可能性があります。

保存の仕方	データベースに入る値	中身が外に出たとき
入力されたまま保存	<code>Passw0rd!</code>	読んだ人がそのままログインできる
bcrypt で変換して保存	<code>\$2b\$10\$...</code> (60文字前後)	元のパスワードは復元できない

Day 06: ユーザー登録画面を作ろう

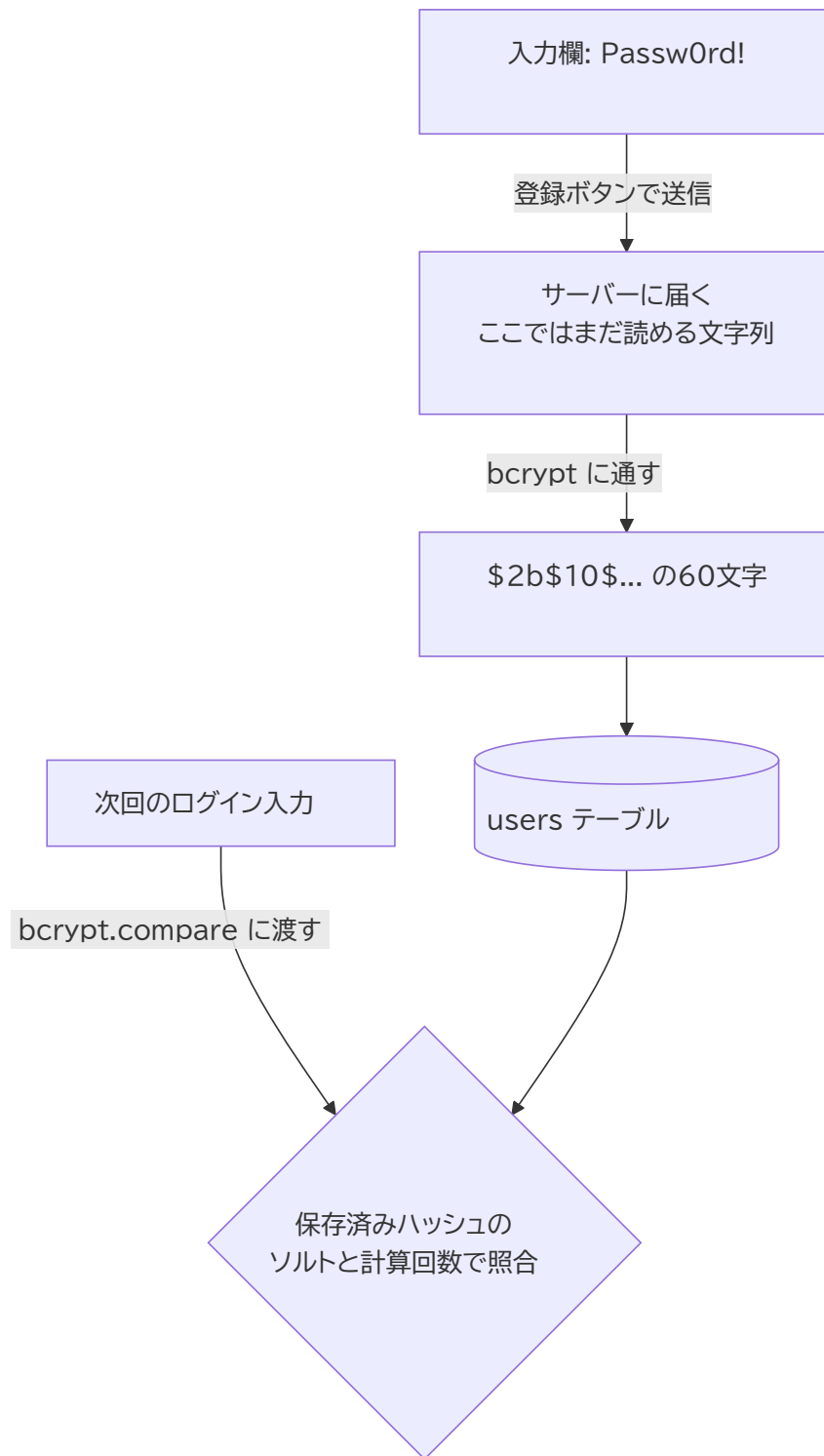


図 5: 送ったパスワードはサーバーでどうなるか

図に `$2b$10$...` から元のパスワードへ戻る矢印がないのは、復元する手順がないためです。ログインでは `bcrypt.compare` が保存済みハッシュのソルトと計算回数を使い、入力合うかを確認めます。ただし保存されたハッシュが漏れると、推測した候補を同じ手順に通して一致するか試される可能性は残ります。

パスワードを画面から送るところまでが今日の範囲で、受け取った側がそれをどう扱うかは Day 07 で自分の手で書きます。このコードは送信値を別の state やログへ保存しません。ただし送信までの入力値はフォーム内にあり、ブラウザのパスワード保存機能に残る場合もあります。

Step 8: アイコンとカード見出しを更新する(読む目安: 3分)

ゴール: カードヘッダーにアイコンと説明文を追加します。

実装:

Day 06: ユーザー登録画面を作ろう

`return` 文の中にある `<CardHeader>` を以下のコードに丸ごと書き換えます。

```
{/* filepath: src/app/register/page.tsx */}
{/* 既存の <CardHeader>...</CardHeader> を書き換え */}
<CardHeader
  className="space-y-1 text-center">
  <div className="flex justify-center mb-2">
    <div className="rounded-full
      bg-gradient-to-r from-blue-500
      to-indigo-500 p-3 shadow-lg">
      <UserPlus className="h-6 w-6
        text-white" />
    </div>
  </div>
  <CardTitle className="text-2xl">
    新規登録
  </CardTitle>
  <CardDescription>
    新しいアカウントを作成してください
  </CardDescription>
</CardHeader>
```

`<CardHeader>` を丸ごと書き換えるのは中身を足すだけでは配置が決まらないからです。`text-center` を親に付けるとその下のタイトルと説明文がまとめて中央へそろいます。アイコンだけは `flex justify-center` を持つ `<div>` で別に包みます。文字と違って画像は `text-center` では中央に寄らないためです。

`UserPlus` に人型のアイコンを選ぶとこの画面が「新しい人を登録する場所」だと文字を読む前に伝わります。ログイン画面の鍵アイコンと並べたとき読者は自分がどちらに居るのかを一目で判断できます。

確認ポイント:

- 人型アイコンが表示されています。
- 「新しいアカウントを作成してください」と表示されています。

ログイン画面と同じグラデーション

(`bg-gradient-to-r from-blue-500 to-indigo-500`) をアイコン背景に使っています。認証系ページで統一感のあるデザインを実現しています。

Step 9: エラー表示と送信ボタンを追加する(読む目安: 5分)

ゴール: サーバーエラーの表示と送信ボタンを追加します。

実装:

`<form>` タグの直後(最初の入力欄の前)にサーバーエラー表示を追加します。

```
{/* filepath: src/app/register/page.tsx */}
{/* <form ...> の直後、名前入力欄の前に追加 */}
{error && (
  <Alert variant="destructive">
    <AlertCircle className="h-4 w-4" />
    <AlertTitle>エラー</AlertTitle>
    <AlertDescription>
      {error}
    </AlertDescription>
  </Alert>
)}
```

この帯が出るのは `error` に文字列が入っているときだけです。Step 7 で書いた `onError` が `setError` を呼ぶと React が画面を描き直し、ここが現れます。

`error` に入るのはメールアドレスの重複のようにサーバーにしか判断できない失敗です。画面側の zod はデータベースの中身を知らないなのでこの種類の失敗は先に見つけられません。もしこの表示を置かないと読者には「登録ボタンを押しても何も起きない」画面だけが残ります。入力を疑って何度も打ち直すことになり、本当の原因である重複には永久にたどり着けません。

確認ポイント:

- ファイルを保存しました。
- `npm run dev` でエラーが出ていません。

次にパスワード確認欄の `</div>` の後(`</form>` の前)に送信ボタンを追加します。

```

{/* filepath: src/app/register/page.tsx */}
{/* パスワード確認欄の後、</form> の前に追加 */}
<Button
  type="submit"
  className="w-full bg-gradient-to-r
    from-blue-600 to-indigo-600
    hover:from-blue-700
    hover:to-indigo-700 shadow-md"
  disabled={registerMutation.isPending}>
  {registerMutation.isPending
    ? '登録中...'
    : '登録'}
</Button>

```

このブロックでいちばん効いているのは

`disabled={registerMutation.isPending}` です。`isPending` は通信している間だけ `true` になり、その間ボタンは押せません。ここを外すと返事が遅いときに読者がボタンを連打し、同じ内容の登録リクエストが並んで飛びます。2件目以降は「このメールアドレスは登録済みです」で弾かれるので1件目が成功しているのに画面には赤いエラーが出る、という読み解けない状態になります。

ラベルを「登録中...」へ差し替えているのも同じ理由です。押せないことと、いま待っていることがボタンの見た目だけで伝わります。`type="submit"` を付けているのでこのボタンは `<form>` の `onSubmit` を呼びます。だからクリック用の関数をここに書く必要はありません。

Day 06: ユーザー登録画面を作ろう

確認ポイント:

- 「登録」ボタンが表示されています。
- 空のまま押すと名前欄とメール欄の下にエラーメッセージが出ます。
- パスワードを7文字で押すと文字数のエラーが出ます。
- 2つのパスワードを食い違わせて押すと「パスワードが一致しません」がパスワード(確認)欄の下に出ます。

Step 4 から Step 6 で書いた検証はここで初めて動かします。送るきっかけになるボタンがこの Step でそろったためです。

Step 10: ログインへのリンクを追加して完成(読む目安: 3分)

ゴール: ログインページへのリンクを追加して登録画面を完成させます。

実装:

`<Button>` の直後(`</form>` の前)にリンクを追加します。

```

{/* filepath: src/app/register/page.tsx */}
{/* Buttonの後、</form> の前に追加 */}
<div className="text-center text-sm
  text-muted-foreground">
  すでにアカウントをお持ちの方は{' '}
  <Link
    href="/login"
    className="text-blue-600 underline
      underline-offset-4
      hover:text-blue-800">
    ログイン
  </Link>
</div>

```

`{' '}` は日本語の文と `<Link>` の間へ半角スペースを1つ入れる書き方です。JSX はタグの前後にある改行とインデントを詰めて出力するのでこれを書かないと「方は」と「ログイン」がくっついて表示されます。

`<a href="/login">` ではなく `<Link>` を使うのはページ全体を読み込み直さずにログイン画面へ切り替えるためです。

確認ポイント:

- 「すでにアカウントをお持ちの方は」リンクが表示されます。
- リンクをクリックするとログインページに遷移します。
- 登録成功でログイン画面へ移り、「登録が完了しました」と表示されます。

Day 06: ユーザー登録画面を作ろう

いま登録したアカウントは一般ユーザーで、初期データのプロジェクトには参加していません。初期データとはDay 01 の初期セットアップで「シードデータを投入しています...」と表示されたときにデータベースへ入った、練習用のユーザーとプロジェクトのことです。自分で作った覚えがなくてもそのときに `admin@example.com` というアカウントとプロジェクトが2つ作られています。いま登録したアカウントはそのプロジェクトのメンバーではないのでそのままだと Day 09 のプロジェクト一覧が空になります。

そこで Day 07 からは `admin@example.com` でログインし直して進めます。ブラウザのアドレス欄に `http://localhost:3000/login` と直接入力し、`admin@example.com` / `password123` でログインしてください。ログアウトのボタンを作るのは Day 08 なのでそれまではこの方法で切り替えます。

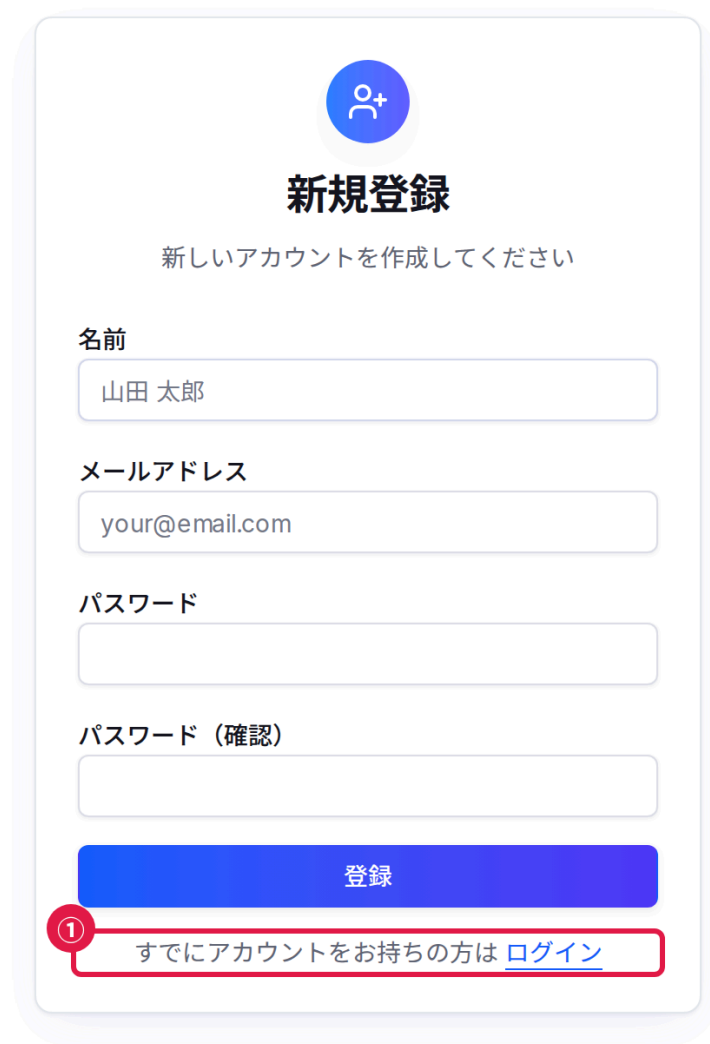
ログインできない場合は、付録のトラブルシューティング^{*1}を開いて「ログインできない」の見出しを探してください。応答のエラーとアカウントの有無を順番に確認します。データベース接続などのエラーや入力間違いは、シードの再実行では直りません。

初期アカウントの `admin@example.com` が無いと確認できた場合だけ、開発サーバーとは別のターミナルで `npm run db:seed` を実行します。このコマンドは初期データの2プロジェクトを削除して作り直すため、その中へ追加したタスクとコメントも消えます。表示された削除対象を確認し、残したい内容がある場合は `n` で中止してください。アカウントがすでにある場合は再実行しません。既存ユーザーのパスワードは入れ直されないためです。

1. <https://drive.google.com/file/d/1OyjFWiGvZBm9QKcnKiougPZidjsn7kqR/view?usp=drivesdk>

`password123` が今日 Step 2 で決めた「大文字・小文字・数字・記号をそれぞれ1つ以上」という規則を満たしていないことに気づいた人もいるはずです。矛盾ではありません。あの規則は登録画面が新しいパスワードを受け取るときの関門で、ログイン画面ではそのチェックをしません。ログイン側は入力が入力欄に空でないかだけを見てあとは保存済みのものと一致するかを照合します。規則を厳しくしたあとで既存の利用者がログインできなくなる事態を避けるためです。`admin@example.com` は登録画面を通らずに初期データとして直接作られたアカウントなので登録時の規則も通っていません。

スクリーンショット: デザインが整った登録画面の完成形です。



新規登録

新しいアカウントを作成してください

名前

山田 太郎

メールアドレス

your@email.com

パスワード

パスワード (確認)

登録

① すでにアカウントをお持ちの方は [ログイン](#)

図6: 完成後の新規登録画面。赤枠の中に「すでにアカウントをお持ちの方は ログイン」のログインリンクが出ている

Pro パターンで書こう(登録APIの結果は判別共用体で受け取る)

成功と失敗の種類を判別共用体(成功・失敗を示す共通のキーで分岐できる型)で表すと分岐の読み間違いをコンパイルエラーとして検出できます。なぜ判別共用体で表すのかを、**Before/After** で見比べてみましょう。

Before(改善前のコード)

```
type RegisterInput = {  
  name: string;  
  email: string;  
  password: string;  
};  
  
type RegisteredUser = {  
  id: string;  
  name: string;  
  email: string;  
};  
  
type RegisterApiResponse = RegisteredUser | string;  
  
const registeredEmails = new  
Set<string>(['admin@example.com']);  
  
export function registerUser(input: RegisterInput):  
RegisterApiResponse {  
  if (registeredEmails.has(input.email)) {  
    return 'このメールアドレスは登録済みです';  
  }  
  
  return {  
    id: 'user_001',  
    name: input.name,
```

Day 06: ユーザー登録画面を作ろう

目を留めてほしいのは

```
type RegisterApiResponse = RegisteredUser | string; の行です。
```

文字列なら失敗、オブジェクトなら成功、という取り決めが型のどこにも書かれていません。この取り決めは書いた人の頭の中にしか無いので半年後に読み返した自分や、引き継いだ人が取り違えます。

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。

```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)

    email: input.email,
  };
}

export function buildRegisterMessage(result:
RegisterApiResponse): string {
  if (typeof result === 'string') {
    return result;
  }

  return `${result.name}さんの登録が完了しました`;
}

const result = registerUser({
  name: 'Taro',
  email: 'kouiso@example.com',
  password: 'password123',
});

console.log(buildRegisterMessage(result));
```

`typeof result === 'string'` は、文字列なら失敗、オブジェクトなら成功という約束に従って値を振り分けています。TypeScript もこの条件から型を絞り、分岐内で使えるプロパティを確かめます。ただ、型だけでは文字列が失敗を表すという意味までは伝わりません。成功・失敗を示す項目を明示すると、呼び出し側が返り値の形から意味を推測せずに済みます。

Day 06: ユーザー登録画面を作ろう

このコードの問題点:

- `string` なら失敗、`object` なら成功という約束が型名から読み取りづらいです。
- エラーにコードやフィールド名を足したくなったとき `string` では表現が足りなくなります。
- 呼び出し側が毎回 `typeof` を覚えておく必要があり、分岐の書き方が揺れやすいです。

After(プロが書くコード)

```
type RegisterInput = {
  name: string;
  email: string;
  password: string;
};

type RegisteredUser = {
  id: string;
  name: string;
  email: string;
};

type RegisterResult =
  | { ok: true; user: RegisteredUser }
  | { ok: false; error: string };

const registeredEmails = new
Set<string>(['admin@example.com']);

export function registerUser(input: RegisterInput):
RegisterResult {
  if (registeredEmails.has(input.email)) {
    return {
      ok: false,
      error: 'このメールアドレスは登録済みです',
    };
  };
```

Day 06: ユーザー登録画面を作ろう

`RegisterResult` がこの書き換えの中心です。 `{ ok: true; user: ... }` と `{ ok: false; error: ... }` の2つの形を `|` で並べてあります。成功のときだけ `user` があり、失敗のときだけ `error` があります。どちらの形なのかを `ok` という1つのキーだけで見分けられるのでこれを判別共用体と呼びます。

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。

```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)

}

return {
  ok: true,
  user: {
    id: 'user_001',
    name: input.name,
    email: input.email,
  },
};
}

export function buildRegisterMessage(result:
RegisterResult): string {
  if (!result.ok) {
    return result.error;
  }

  return `${result.user.name}さんの登録が完了しました`;
}

const result = registerUser({
  name: 'Taro',
  email: 'kouiso@example.com',
  password: 'password123',
```

Day 06: ユーザー登録画面を作ろう

`buildRegisterMessage` の `if (!result.ok)` を書いた次の行では TypeScript が `result` を失敗側の形だと決めてくれます。だから `result.error` はそのまま読め、そこで `result.user` と書けば即座に赤線が出ます。分岐を1本書いた瞬間に、その先で読める項目まで決まります。Before 側で必要だった `typeof` の確認はもう出てきません。

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。

```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)
});

console.log(buildRegisterMessage(result));
```

呼び出す側は `registerUser` の中身を読まなくても `ok` を1つ見れば次にどの項目を読めばよいか決まります。今日の `registerMutation` が `onSuccess` と `onError` で処理を分けているのも、同じ考え方です。成功と失敗を1つの入れ物に混ぜず、初めから別の道に分けておきます。

このコードの強み:

- `ok` を見るだけで成功と失敗が分かれ、返ってくる値の形も TypeScript が絞って入ってくれます。
- 失敗時に `errorCode` や `field` を足す場合も、失敗側の型へ自然に拡張できます。
- 呼び出し側の分岐が毎回同じ形になるので登録後の画面遷移やエラー表示を保守しやすいです。

覚えておきたいエッセンス

APIレスポンスは「なんとなく文字列なら失敗」より成功と失敗の形を型で分けるほうが強いです。判別共用体にしておくと分岐を書いた瞬間に中身の型まで決まるのです。

完成コード全体

途中で迷った場合は以下のコードをそのまま `src/app/register/page.tsx` にコピーしてください。

import 部分:

```
// filepath: src/app/register/page.tsx
// 完成版: import部分
'use client';

import { zodResolver } from '@hookform/resolvers/zod';
import { AlertCircle, UserPlus } from 'lucide-react';
import Link from 'next/link';
import { useRouter } from 'next/navigation';
import { useState } from 'react';
import { useForm } from 'react-hook-form';
import { z } from 'zod';
```

1枚目は外部ライブラリの取り込みです。並びがアルファベット順になっているのは `npm run fix` を実行すると Biome(このプロジェクトのコード整形ツール)が並べ替えるからです。自分が書いた順番と違っていても手で直す必要はありません。

Day 06: ユーザー登録画面を作ろう

```
// filepath: src/app/register/page.tsx
// 完成版: shadcn/ui と trpc の import
import {
  Alert, AlertDescription, AlertTitle,
} from '@component/ui/alert';
import { Button } from '@component/ui/button';
import {
  Card, CardContent,
  CardDescription, CardHeader, CardTitle,
} from '@component/ui/card';
import { Input } from '@component/ui/input';
import { Label } from '@component/ui/label';
import { api } from '@trpc/react';
```

2枚目は自分のプロジェクトの中にある部品です。`@/` で始まる書き方は `src/` を指す近道で、ファイルの置き場所が深くなっても `../..` を数えずに済みます。

`@/component/ui/...` が単数形になっている点は Step 1 の注意書きのとおりです。ここを複数形で書くとそんなファイルは無いというエラーが起動時に出了。

zod スキーマと型定義:

```
// filepath: src/app/register/page.tsx
// 完成版: バリデーションスキーマ
const registerSchema = z.object({
  name: z.string()
    .min(1, '名前を入力してください'),
  email: z.string()
    .email('有効なメールアドレスを入力してください'),
  password: z.string()
    .min(8, 'パスワードは8文字以上で入力してください')
    .regex(/[A-Z]/,
      'パスワードに大文字を1文字以上含めてください')
    .regex(/[a-z]/,
      'パスワードに小文字を1文字以上含めてください')
    .regex(/[0-9]/,
      'パスワードに数字を1文字以上含めてください')
    .regex(/[^\A-Za-z0-9]/,
      'パスワードに記号を1文字以上含めてください'),
  confirmPassword: z.string()
    .min(1, 'パスワード(確認)を入力してください'),
}).refine(
  (data) => data.password === data.confirmPassword,
  {
    message: 'パスワードが一致しません',
    path: ['confirmPassword'],
  }
)
```

この `registerSchema` が登録画面の入口の検査を1か所に集めた部分です。4本の `.regex()` は Day 07 で読むサーバー側と同じ文字種の条件です。サーバー側が追加で検査する72バイトの上限は、今日のスキーマには入っていません。文字数と

Day 06: ユーザー登録画面を作ろう

バイト数は別で、日本語の「あ」は UTF-8 では3バイトです。長すぎる入力を送ると、サーバーから返るエラーを Step 9 の赤い帯で確認できます。このブロックはまだ括弧が開いたままなので次の続きと合わせて1つの式になります。

```
// filepath: src/app/register/page.tsx(同じファイルの続き)
},
);

type RegisterFormData =
  z.infer<typeof registerSchema>;
```

`.refine()` の閉じ括弧と `RegisterFormData` の型定義までがコンポーネント関数の外に置く部分です。ここを外に出しておくとスキーマは画面が描き直されるたびに作り直されません。関数の中へ入れても動きますが入力のたびに同じルールを組み立て直すことになります。

コンポーネント本体(ロジック部分)は次のとおりです。

```
// filepath: src/app/register/page.tsx
// 完成版: RegisterPage関数の前半
export default function RegisterPage() {
  const router = useRouter();
  const [error, setError] =
    useState<string | null>(null);
  const {
    register, handleSubmit,
    formState: { errors },
  } = useForm<RegisterFormData>({
    resolver: zodResolver(registerSchema),
  });
```

Day 06: ユーザー登録画面を作ろう

関数の前半は道具をそろえる部分です。`router` は登録後の移動先を指示する道具、`error` はサーバーから返った文言を覚えておく場所、`useForm` の3つはフォーム本体です。定義の順番がこの形になっているのは次のブロックの `registerMutation` が `router` と `setError` を使うからです。使う側より先に用意しておく必要があります。

```
// filepath: src/app/register/page.tsx
// 完成版: mutation と onSubmit

const registerMutation =
  api.auth.register.useMutation({
    onSuccess: () => {
      router.push('/login?registered=1');
    },
    onError: (error) => {
      setError(
        error.message
        ?? 'ユーザー登録中にエラーが発生しました'
      );
    },
  });

const onSubmit = async (
  data: RegisterFormData
) => {
  setError(null);
  registerMutation.mutate({
    name: data.name,
```

```
    email: data.email,  
    password: data.password,  
  });  
};
```

後半は通信と送信処理です。`onSubmit` が呼ばれるのは `handleSubmit` が zod の検査を全部通した後だけなのでこの中で入力値を確かめ直す必要はありません。

`mutate` へ渡す 3 項目に `confirmPassword` が入っていない点も、ここで見ておきます。手元のコードで 4 項目になっていたらその差が「型エラーが出る」というつまりの正体です。

JSX(表示部分)は次のとおりです。

Day 06: ユーザー登録画面を作ろう

```
// filepath: src/app/register/page.tsx
// 完成版: return文 - CardHeaderまで
return (
  <div className="flex min-h-screen
    items-center justify-center px-4">
    <Card className="w-full max-w-sm">
      <CardHeader
        className="space-y-1 text-center">
        <div className="flex justify-center mb-2">
          <div className="rounded-full
            bg-gradient-to-r from-blue-500
            to-indigo-500 p-3 shadow-lg">
            <UserPlus className="h-6 w-6
              text-white" />
          </div>
        </div>
      </div>
      <CardTitle className="text-2xl">
        新規登録
      </CardTitle>
      <CardDescription>
        新しいアカウントを作成してください
      </CardDescription>
    </CardHeader>
```

`return` の前半は画面の外枠とカードの見出しです。`min-h-screen` で高さを画面いっぱいに取り、`items-center justify-center` で縦横の中央にカードを置きます。`max-w-sm` があるので横に広いモニターでもカードは間延びしません。こまでは Step 1 と Step 8 で書いた内容がそのまま並んでいます。

```

{/* filepath: src/app/register/page.tsx */}
{/* 完成版: CardContent - 名前・メール欄 */}
<CardContent>
  <form
    onSubmit={handleSubmit(onSubmit)}
    className="space-y-4">
    {error && (
      <Alert variant="destructive">
        <AlertCircle className="h-4 w-4" />
        <AlertTitle>エラー</AlertTitle>
        <AlertDescription>
          {error}
        </AlertDescription>
      </Alert>
    )}
    <div className="space-y-2">
      <Label htmlFor="name">名前</Label>
      <Input id="name" type="text"
        placeholder="山田 太郎"
        autoComplete="name" autoFocus
        {...register('name')} />
      {errors.name && (
        <p className="text-sm text-destructive">
          {errors.name.message}</p>)}
    </div>
  </form>

```

Day 06: ユーザー登録画面を作ろう

`<CardContent>` の中身は `<form>` ひとつです。

`onSubmit={handleSubmit(onSubmit)}` という二重の呼び出しがこの画面の要になります。外側の `handleSubmit` が先に走って zod の検査をし、全部通ったときだけ内側の `onSubmit` へ値を渡します。ここを `onSubmit={onSubmit}` へ直接つないだ場合検査は飛ばされ、空欄のままサーバーに届きます。

サーバーエラーの `<Alert>` を入力欄より前に置いているのは失敗したときに読者の目が最初に届く位置だからです。ボタンの下に置くと入力欄が長い画面では画面外に隠れます。

```
{/* filepath: src/app/register/page.tsx */}
{/* 完成版: メール・パスワード欄 */}
<div className="space-y-2">
  <Label htmlFor="email">
    メールアドレス</Label>
    <Input id="email" type="email"
      placeholder="your@email.com"
      autoComplete="email"
      {...register('email')} />
    {errors.email && (
      <p className="text-sm text-destructive">
        {errors.email.message}</p>)}
  </div>
  <div className="space-y-2">
    <Label htmlFor="password">
      パスワード</Label>
    <Input id="password" type="password"
      autoComplete="new-password"
      {...register('password')} />
    {errors.password && (
      <p className="text-sm text-destructive">
        {errors.password.message}</p>)}
  </div>
```

Day 06: ユーザー登録画面を作ろう

メール欄とパスワード欄では `register` に渡す名前と `errors` から読むキーが対になっています。`register('email')` と `errors.email`、
`register('password')` と `errors.password` のように必ず同じ名前で組にします。片方だけ書き換えると打ち間違えた欄とは違う場所に赤い文字が出ます。読者はエラーの出ていない欄を直そうとして時間だけを使います。

```
{/* filepath: src/app/register/page.tsx */}
{/* 完成版: パスワード確認欄 */}
<div className="space-y-2">
  <Label htmlFor="confirmPassword">
    パスワード(確認)</Label>
    <Input id="confirmPassword"
      type="password"
      autoComplete="new-password"
      {...register('confirmPassword')} />
    {errors.confirmPassword && (
      <p className="text-sm text-destructive">
        {errors.confirmPassword.message}
      </p>)}
</div>
```

確認欄も書き方は同じですがここに出るメッセージだけは出どころが違います。他の3欄は自分の入力を見て判定されるのに対し、この欄の「パスワードが一致しません」は `.refine()` が2つの欄を見比べた結果です。`path` の指定でこの場所へ届いている、という関係をもう一度確かめておきます。

```

    { /* filepath: src/app/register/page.tsx */ }
    { /* 完成版: ボタン・リンク・閉じタグ */ }

    <Button type="submit"
      className="w-full bg-gradient-to-r
from-blue-600 to-indigo-600
      hover:from-blue-700 hover:to-indigo-700
shadow-md"
      disabled={registerMutation.isPending}>
      {registerMutation.isPending
        ? '登録中...' : '登録'}
    </Button>

    <div className="text-center text-sm
text-muted-foreground">
      すでにアカウントをお持ちの方は{' '}
      <Link href="/login"
        className="text-blue-600 underline
        underline-offset-4 hover:text-blue-800">
        ログイン</Link>
    </div>
  </form>
</CardContent>
</Card>
</div>
);
}

```

Day 06: ユーザー登録画面を作ろう

最後は送信ボタンとログインへのリンク、そして開いていたタグを内側から順に閉じる部分です。`</form>`、`</CardContent>`、`</Card>`、`</div>` という順番が上で開いた順番の逆になっていることを確かめてください。1つでも抜けるとブラウザに赤いエラー画面が出ます。同じ内容は開発サーバーのターミナルにも表示されます。どちらにも、閉じタグの足りない場所が書かれています。

完成形の参考コード: 完成版には `src/app/register/page.tsx` があります。ただし今日書いたコードと1文字まで同じではありません。違うのは次の4か所です。1つ目は完成版がパスワード条件を共通のヘルパー関数 (`createPasswordSchema`) から読み込んでおり、大文字・小文字・数字・特殊文字に加えて、bcrypt の上限である UTF-8 で 72 バイト以内の検査を含んでいる点です。2つ目はエラーの入れ物が `useState<string[]>([])` になっていてサーバーから返った複数のメッセージを並べて出せる形になっている点です。3つ目は完成版の入力欄に `aria-required` や `aria-invalid` といった読み上げ用の指定が付いている点です。読み上げソフトへの対応は Day 10 でまとめて扱います。4つ目は完成版の背景に `blur-3xl` のぼかし装飾があり、色を `bg-primary` などのテーマ名で指定していて必須の入力欄に `text-destructive` の赤い印が付いている点です。今日は色を `from-blue to-indigo` と直に書いています。登録後の移動先と固定の完了案内は完成版と同じです。(販売用 ZIP に完成版の `src/` は入っていません。ここに挙げた違いは完成版がどう書かれているかの説明として読んでください)。

Day 05(ログイン)と Day 06(登録)の比較

項目	Day 05 ログイン	Day 06 登録
アイコン	<code>Lock</code> (鍵)	<code>UserPlus</code> (人型+)
アイコン背景色	グラデーション (blue→indigo)	グラデーション (blue→indigo)
フィールド数	2つ(email, password)	4つ(name, email, password, confirmPassword)
バリデーション	基本チェックのみ	<code>.refine()</code> でクロス チェック
トースト通知	<code>react-hot-toast</code> で成 功通知	なし(ログイン画面の固定 案内で完了を伝えるため 省略)
Suspense	必要(<code>useSearchParams</code> 使用)	不要(<code>useSearchParams</code> 未使用)

完成コードを書き終えたらターミナルで `npm run type-check` を実行してください。型のエラーが出た場合は表示されたファイル名と行番号を確認して直します。画面の動作は本文の操作手順で別に確かめます。

今日のまとめ

- Day 05 のパターンを応用して登録フォームを作れました。
- `.refine()` でパスワード一致チェックを実装できました。
- 4つの入力フィールドを `react-hook-form` で管理できました。

Day 06: ユーザー登録画面を作ろう

- ☐ `confirmPassword` を除外してAPIに送信できました。
- ☐ ローディング・エラー表示を実装できました。
- ☐ 登録後に固定の完了案内を出してログインへ移せました。

つまずきポイント

パスワード一致エラーが出ない

原因

`.refine()` の `path` 指定が漏れています。

解決方法

`path: ['confirmPassword']` を追加します。

登録後にログイン画面へ移らない

原因

登録成功時の移動先が古いままか、`router.push` を呼び忘れてしています。

解決方法

`onSuccess` の中に `router.push('/login?registered=1')` があるか確認めます。ログイン画面で案内が出ない場合は、`registrationCompleted` が文字列の `'1'` と比較されているかも確認してください。

「このメールは登録済み」エラー

原因

同じメールアドレスで二度登録しています。

解決方法

別のメールアドレスで試します。

型エラーが出る

原因

`confirmPassword` を API に送っています。

解決方法

`mutate` で必要なフィールドだけを指定します。

クライアントでは通るのにサーバーエラーになる

原因

パスワードがサーバー側の条件を満たしていない場合があります。今日の画面では検査していない72バイトの上限も、サーバー側では検査します。通信の失敗でもエラーが出るため、まず赤い帯のメッセージを確認します。

解決方法

「UTF-8で72バイト以内」と表示されたらパスワードを短くして送信し直します。文字種の条件が表示されたら、Step 2 の `registerSchema` に大文字・小文字・数字・記号の4本の `.regex()` がそろっているか確認してください。通信のエラーなら開発サーバーのターミナルにエラーが出ていないか確認し、起動状態を確かめてから送信し直します。

今日学んだ用語

用語	意味
<code>.refine()</code>	複数フィールドにまたがるカスタムバリデーション
<code>path</code>	エラーメッセージの表示先フィールドを指定する
<code>autoComplete</code>	ブラウザの自動入力の種類を指示するHTML 属性
<code>confirmPassword</code>	パスワード確認用のフロントエンド専用フィールド

応用課題: 確認欄のエラーを出す条件を変える

2つの欄を比較する検査を確かめましょう。入力欄を直した後の再送信で、エラーの原因を切り分けます。

(1) `/register` で名前とメールアドレスを入力し、パスワードには `Passw0rd!`、確認欄には `Passw0rd?` を入れます。どちらも文字数と文字種の要件を満たしたうえで値だけが違います。登録を押し、確認欄の不一致エラーを確認します。

(2) 確認欄をパスワードと一致させ、今度は名前を空にしてから登録を押しします。名前を空にするのはこの課題で利用者を新規登録しないためです。

(3) 確認欄の不一致エラーが消え、名前の必須エラーが出れば成功です。不一致が残る場合は2つのパスワードの文字を確かめます。

(4)ページを再読み込みして入力とエラーを消します。ソースコードと登録済みの利用者は変更しません。

理解チェック

今日書いたコードを見ながら答えてみてください。答えは各問のすぐ下にあります。

Q1.

```
.refine((data) => data.password === data.confirmPassword, { path: ['confirmPassword'] })
```

は何をしている検査ですか。

A. 各項目の個別チェックが済んだあとに動く、最後の見比べです。パスワード欄と確認欄の2つを突き合わせ、違っていればエラーにします。1つの欄しか見ない項目チェックでは書けないため、この実装では `.refine()` で行います。`path` はそのエラーをどの欄の下に出すかの指定です。

Q2. その `path: ['confirmPassword']` を書き忘れると画面はどうなりますか。

A. エラーがフォーム全体に付き、`errors.confirmPassword` は空のままになります。どの入力欄の下にも赤い文字が出ません。利用者から見えるのは「登録ボタンを押しても画面が変わらない」という状態だけです。どこを直せばよいかの手掛かりが画面に1つも残りません。

Q3. `mutate` に `confirmPassword` を渡していないのはなぜですか。

A. 確認欄は打ち間違いをその場でを見つけるための画面側だけの欄だからです。サーバーは保存するパスワードを1つ受け取れば足ります。同じ値をもう1つ送ってもサーバー側ですることはありません。送る項目が少ないほど途中で漏れる範囲も狭くなります。

次回予告

Day 07 では今日作った登録画面と Day 05 のログイン画面が実際にどう動いているのか、裏側の仕組みを学びます。jose(JWT HS256)と HTTP-only Cookie を使った認証の流れと、サーバー側でのセッション管理を理解しましょう。

次に読むもの

- 前の日: Day 05^{*2}
- 次の日: Day 07^{*3}
- 全体の地図: 学びのロードマップ^{*4}
- 目次: カリキュラム目次^{*5}
- 詰まったとき: トラブルシューティング^{*6}
- 言葉の意味: 用語集^{*7}

奥付

タイトル	Day 06: ユーザー登録画面を作ろう

2. <https://drive.google.com/file/d/18Mq1zmBcLFNfxATqwHoeFrtzeGldT4UZ/view?usp=drivesdk>

3. <https://drive.google.com/file/d/13-JuMAvw7EsJJJMYgyk5HayoeBiGqkTk/view?usp=drivesdk>

4. https://drive.google.com/file/d/10py_p-6ry4RDlxBSyzPn4rtzZe74ND-T/view?usp=drivesdk

5. <https://drive.google.com/file/d/19g0cKLajEWi7s9LeWF804DB9j5kK1dt4/view?usp=drivesdk>

6. <https://drive.google.com/file/d/1OyjFWiGvZBm9QKcnKiougPZidjsn7kqR/view?usp=drivesdk>

7. <https://drive.google.com/file/d/1RtElBpzkvXsoTpj4CTKPko8jnQDcPsh/view?usp=drivesdk>

著者

磯貝光佑

版

第1版(2026年9月19日)

© 2026 磯貝光佑

本書は購入者個人の利用に限ります。本書に掲載されたコードの写経、および自分のプロジェクトへの転用・改変は自由です。本書(PDF・Markdown)および付属コードの再配布・転売・第三者との共有は、形態を問わず禁止します。